

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря Сікорського”
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ
Кафедра інформаційної безпеки

Лабораторна робота №1
з дисципліни «Технологія блокчейн та розподілені
системи»

Тема: „Розгортання систем Ethereum та криптовалют”.

Мета роботи: «Отримання навичок налаштування платформ виконання смартконтрактів та криптовалют».

Завдання: Провести порівняльний аналіз особливостей розгортання систем криптовалют у порівнянні із системою Ethereum. Зробити висновок про можливість чи неможливість взаємозаміни модулів різних систем та пояснити причини.

Виконав: ФБ-51мн Тимощук Ілья

Київ 2026 р.

ЗМІСТ

ВСТУП.....	3
1 АРХІТЕКТУРНІ МОДЕЛІ ТА ПРИНЦИПИ ФУНКЦІОНУВАННЯ	4
1.1. Класифікація систем: Ethereum проти класики та нішевих платформ	4
1.2. Порівняння моделей стану: Accounts проти UTXO.....	5
1.3. Стек технологій: як мова ядра впливає на розгортання вузла та нерви аналітика.....	6
2 ОСОБЛИВОСТІ РОЗГОРТАННЯ ТА КОНФІГУРАЦІЇ ВУЗЛІВ.....	10
2.1. Системні вимоги: де зазвичай «вузьке місце»?.....	10
2.2. Процес ініціалізації приватної мережі: від генезис-блоку до перших транзакцій.....	11
2.3. Мережева топологія: як вузли шукають один одного	13
3 КОМПОНЕНТНИЙ АНАЛІЗ ТА МЕХАНІЗМИ КОНСЕНСУСУ	15
3.1. Модуль консенсусу: технічне порівняння PoW, PoS та dBFT.....	15
3.2. Віртуальні машини та середовища виконання.....	16
3.3. Порівняння характеристик та аналіз ключових метрик.....	18
4 АНАЛІЗ ВЗАЄМОЗАМІННОСТІ МОДУЛІВ ТА ІНТЕГРАЦІЇ	21
4.1. Гнучкість модульної архітектури	21
4.2. Інтерфейси програмування (API) та інтеграція: від JSON-RPC до нативних оракулів	22
4.3. Аналіз масштабованості та оновлюваності: форки, управління та вектори розвитку.....	23
5 ВИСНОВКИ ТА СТРАТЕГІЧНІ РЕКОМЕНДАЦІЇ ЩОДО ВИБОРУ ПЛАТФОРМИ	26
5.1. Підсумкова оцінка технологічної придатності	26
5.2. Стратегічна матриця вибору для системного аналітика	26
5.3. Рекомендації щодо впровадження та мінімізації ризиків	27

ВСТУП

Технології розподіленого реєстру (DLT) розвиваються настільки стрімко, що ми вже маємо справжній зоопарк архітектурних підходів. Кожен із них, по суті, намагається розв'язати ту саму задачу – «трилему блокчейну»: як знайти ідеальний баланс між децентралізацією, безпекою та швидкістю роботи. Для системного аналітика це не просто теорія. Розуміння того, як ці системи влаштовані «під капотом», є критичним, адже від обраної платформи залежить усе: від швидкості транзакцій до того, скільки болю принесе розгортання інфраструктури та скільки це зрештою коштуватиме в експлуатації.

Ця лабораторна робота присвячена комплексному аналізу того, як розгортаються та функціонують блокчейни різних поколінь. У центрі уваги – платформа Ethereum, такий собі еталон для систем із підтримкою Тюрінг-повних смарт-контрактів. Ми порівняємо її з класикою платіжних мереж (Bitcoin, Litecoin, Dash) та більш специфічними корпоративними гравцями на кшталт NEO. Важливо дослідити не лише технічні вимоги до вузлів чи тонкощі запуску приватних мереж, а й фундаментальне питання: чи можлива взагалі модульна взаємозамінність компонентів? Отримані результати допоможуть сформулювати тверезі технічні висновки про те, наскільки гнучкими є сучасні децентралізовані рішення.

1 АРХІТЕКТУРНІ МОДЕЛІ ТА ПРИНЦИПИ ФУНКЦІОНУВАННЯ

Почнемо з того, що розберемо блокчейн на за частини. Зазвичай ми дивимося на нього як на складну систему, що складається з мережевого рівня, рівня консенсусу та рівня стану. Навіщо це потрібно? Тільки так можна ідентифікувати точки дотику та побачити, де різні протоколи тиснуть один одному руки, а де розходяться назавжди.

1.1. Класифікація систем: Ethereum проти класики та нішевих платформ

Для аналітика важливо не просто кидати все в один кошик із написом «криптовалюти», а чітко розрізняти системи за їхньою архітектурою та роллю у цифровому стеку.

1. Ethereum: «Світовий комп'ютер» та його парадигма

Ethereum – це друге покоління блокчейнів, де основний акцент змістився з простого переказу грошей на виконання програмного коду. З точки зору архітектури це транзакційна машина станів. Якщо Bitcoin можна описати як розподілений грошовий реєстр, то Ethereum працює як величезний розподілений комп'ютер, де кожен вузол мережі підтримує ідентичну копію віртуальної машини (EVM).

- **Щодо розгортання:** На відміну від Bitcoin, де вузол переважно перевіряє підписи та суми, вузол Ethereum має забезпечувати середовище для складних обчислень. Примітно, що для стабільності UNIX-подібних процесів на Windows часто доводиться використовувати WSL при роботі з клієнтами на кшталт Geth (Go-Ethereum).

2. Bitcoin, Litecoin та Dash: Еволюція платежів

Ці системи базуються на кодовій базі Bitcoin Core. Тут головний пріоритет – безпека та стійкість до цензури.

- **Bitcoin** залишається консервативним стандартом. Робота з ним – це переважно взаємодія через інтерфейс командного рядка (CLI) для керування демоном bitcoind та гаманцем bitcoin-cli.
- **Litecoin та Dash** – це, по суті, технологічні надбудови. Litecoin пропонує швидшу генерацію блоків та алгоритм Scrypt, що змушує серйозніше ставитися до оперативної пам'яті сервера.
- **Dash** впроваджує цікаву інновацію – дворівневу мережу. Окрім звичайних майнерів, тут існують Мастерноди для сервісів PrivateSend та InstantSend. Це, чесно кажучи, додає головного болю при конфігурації: треба враховувати заставні вимоги (collateral) та статичну IP-адресацію для вузлів другого рівня.

3. NEO: Платформа «Розумної економіки»

NEO іноді називають «китайським Ethereum», але її архітектура має свої унікальні риси. Головна відмінність – фокус на цифровізації реальних активів та ідентифікації користувачів.

- **Гнучкість розробки:** NEO підтримує контракти на C#, Java, Python та Go. Це робить її привабливою для корпорацій, які вже мають штат програмістів на цих мовах.
- **Механізм консенсусу:** Замість енерговитратного PoW або сучасного PoS, тут використовується dBFT (Delegated Byzantine Fault Tolerance). При розгортанні аналітик має справу не з анонімними майнерами, а з конкретними «вузлами-консенсусу». Це вимагає зовсім іншого підходу до безпеки та довіри всередині мережі.

1.2. Порівняння моделей стану: Accounts проти UTXO

Для системного аналітика модель стану – це фундамент. Це те, як система «пам'ятає», кому що належить. Хоча в інтерфейсі гаманця ми бачимо просто цифру балансу, механіка в Ethereum та Bitcoin-подібних системах кардинально різниться. Саме тут криється головний бар'єр для взаємозамінності їхніх модулів.

1.2.1. Модель UTXO (Unspent Transaction Output)

Цю архітектуру використовують Bitcoin, Litecoin та Dash. Парадокс у тому, що на рівні протоколу поняття «баланс гаманця» просто не існує. Блокчейн – це лише величезний масив транзакцій.

- **Механіка:** Кожна операція складається з «входів» та «виходів». Щоб надіслати кошти, треба послатися на вихід попередньої транзакції, який ще не був витрачений. Це схоже на розрахунок готівкою: якщо у вас є купюра у 100 одиниць, а ви купуєте товар за 30, ви «витрачаєте» всю купюру, отримуючи 70 одиниць «решти» у вигляді нового UTXO.
- **Що це дає аналітику?**
 - **Паралелізм:** Оскільки транзакції незалежні, їх можна перевіряти паралельно. Це гарна новина для швидкості обробки на багатоядерних системах.
 - **Приватність:** Можна створювати нові адреси для кожної решти, що трохи ускладнює відстеження грошових потоків.
- **Недоліки:** Тут ми впираємося в обмежену логіку. Реалізувати складні умови (смарт-контракти) у такій моделі надзвичайно важко, адже система «не пам'ятає» попередніх станів.

1.2.2. Модель облікових записів (Account-based)

Ethereum та NEO пішли іншим шляхом. Ця модель набагато ближча до традиційних банків чи SQL-баз даних.

- **Механіка:** Стан мережі – це глобальна карта (mapping). Кожній адресі відповідає баланс, код смарт-контракту та дані. Транзакція тут – це не

переміщення «монет-купюр», а інструкція: «відняти X з рахунку А та додати на рахунок Б».

- **Роль Nonce:** Варто зауважити, що для захисту від атак повторного відтворення кожен акаунт має лічильник транзакцій (nonce). Це критичний параметр: транзакції мають йти строго по порядку.
- **Переваги для аналітика:**
 - **Простота розробки:** Програмістам легше працювати з об'єктами «баланс» та «стан». Саме це дозволило розквітнути складним DeFi-протоколам.
 - **Економічність:** Транзакції зазвичай компактніші, бо не треба перелічувати всі входи.
- **Недоліки:** Послідовне виконання. Через необхідність змінювати загальний глобальний стан, транзакції важче обробляти паралельно, що створює «вузьке місце» для масштабованості.

Критерій	UTXO (Bitcoin/Litecoin)	Account-based (Ethereum/NEO)
Об'єкт обліку	Невитрачений вихід	Рахунок (Баланс)
Валідація	Можлива паралельна	Переважно послідовна
Смарт-контракти	Обмежені (Script)	Тюрінг-повні
Розмір транзакції	Залежить від кількості входів	Зазвичай фіксований/компактний

1.2.3. Як модель впливає на залізо та синхронізацію

Те, яку модель ви оберете, прямо б'є по вимогах до серверного обладнання:

1. **Синхронізація Bitcoin (UTXO):** Вузол має індексувати набір усіх невитрачених виходів (UTXO set). Зараз цей набір доволі компактний і спокійно живе в оперативній пам'яті для швидкого доступу.
2. **Синхронізація Ethereum (Accounts):** Тут усе складніше. Вузол мусить оновлювати величезне дерево станів (Merkle Patricia Tree). Це вимагає швидких SSD-накопичувачів з високим показником IOPS. Кожна транзакція – це численні операції читання та запису. Саме тому при розгортанні через Geth критично важливо мати достатньо RAM для кешування цього дерева.

1.3. Стек технологій: як мова ядра впливає на розгортання вузла та нерви аналітика

На перший погляд, вибір мови для реалізації протоколу – це лише справа смаку розробників. Проте для системного аналітика це стратегічне рішення. Воно визначає, як вузол поводитиметься під піковим навантаженням, чи не «потече» пам'ять і наскільки болісною буде підтримка всієї інфраструктури. Примітно, що саме мова програмування часто диктує швидкість синхронізації, про яку так часто сперечаються в спільноті.

1.3.1. C++ та «важка» спадщина Bitcoin (Litecoin, Dash)

Більшість класичних систем, що вирости з коду Bitcoin Core, тримаються за C++. Це логічно: розробникам потрібен максимальний контроль над «залізом» та низькорівневими операціями.

- **Технічні нюанси:** C++ дозволяє витиснути максимум із криптографії та мінімізувати затримки в mempool. Але є й зворотний бік. Для успішної компіляції того ж bitcoind або dashd аналітику доводиться збирати цілий «зоопарк» бібліотек. Варто зауважити, що окрім стандартних libboost та libevent, критично важливою залишається застаріла Berkeley DB 4.8 – без неї про сумісність гаманців можна забути.
- **Чого чекати при розгортанні:** Якщо ви вирішили зібрати систему з вихідного коду на слабкому сервері, готуйтеся – процес може тривати годинами. Ручне керування залежностями в Linux часто перетворюється на «пекло версій». Без контейнеризації такі системи важко назвати гнучкими для хмарного масштабування.

1.3.2. Go (Golang) та домінування Ethereum (Geth)

Ethereum свого часу змінив правила гри не тільки у смарт-контрактах, а й у підході до створення клієнтів. Найпопулярніший із них – Geth – написаний на Go.

- **Плюси для інфраструктури:** Go «з коробки» чудово працює з паралелізмом через горутини (goroutines). Для Ethereum, де потрібно обробляти тисячі одночасних P2P-з'єднань, це ідеальне рішення. Що важливо для аналітика – розгортання Geth проходить набагато гладше завдяки статичній лінковці бінарних файлів. Ніяких проблем із залежностями.
- **Контекст середовища:** Завдяки гнучкості Go, вузли Ethereum легко «приживаються» в WSL. Це дозволяє розробникам на Windows працювати в повноцінному Linux-оточенні, не розгортаючи важкі віртуальні машини. Зручно? Надзвичайно. Особливо коли треба швидко протестувати приватну мережу.

1.3.3. Rust: курс на максимальну безпеку

Нове покоління клієнтів (як-от Nethermind для Ethereum або ядра Polkadot та Solana) дедалі частіше обирає Rust. Ця мова пропонує швидкість C++, але виключає помилки роботи з пам'яттю на рівні компілятора. Що це дає для

розгортання? Надзвичайно стабільні вузли. Вони здатні працювати місяцями без перезавантажень, навіть якщо мережу «штормить» від DDoS-атак.

1.3.4. Багатомовна стратегія NEO

Система NEO обрала свій шлях. Вона підтримує високорівневі мови – C#, Java та Python – не тільки для контрактів, а й для компонентів екосистеми.

- **Корпоративний аспект:** Для розгортання NEO знадобляться відповідні рантайми (наприклад, .NET Core Runtime). Це дозволяє безшовно вписати блокчейн у вже наявну IT-структуру компанії. Проте варто пам'ятати про «Virtual Machine overhead». Додатковий рівень абстракції може трохи сповільнити синхронізацію, якщо порівнювати з нативними C++ клієнтами.

Детальний аналіз стеку технологій

Вибір мови програмування та архітектури клієнта безпосередньо впливає на стабільність вузла та ресурсомісткість процесу розгортання.

Клієнт	Мова	Керування пам'яттю	Складність розгортання	Основні залежності
Geth (Go-Ethereum)	Go	GC (Automatic)	Низька: завдяки статичній лінковці бінарних файлів.	Go runtime (лише для збірки), LevelDB/MDBX.
Bitcoin Core	C++	Manual	Висока: потребує ручного керування великою кількістю бібліотек.	libboost, libevent, Berkeley DB 4.8 (критично для сумісності).
Neo-CLI	C#	GC (Automatic)	Середня: вимагає налаштування специфічних рантаймів.	.NET Core Runtime, рівень плагінів для SQL/NoSQL.

1.3.5. Короткі висновки щодо стеку

Вимальовується чітка закономірність: системи першого покоління (Bitcoin-family) жертвують зручністю заради близькості до «заліза». Платформи ж другого та третього поколінь фокусуються на швидкості запуску. Який висновок для аналітика? При проектуванні приватної мережі на базі Bitcoin Core закладайте вдвічі більше часу на налагодження середовища, ніж при роботі з Ethereum.

Порівняння процесів білду

Різниця між C++ та сучасними мовами на кшталт Go або Rust полягає у складності ланцюжка залежностей та способі фінальної збірки додатка. У той час

як C++ вимагає тривалої лінковки зовнішніх бібліотек, Go та Rust частіше пропонують майже готові "самодостатні" рішення.

- **Для C++ (Bitcoin Core):** Шлях включає препроцесор, компілятор, створення об'єктних файлів та складний етап лінковки з системними бібліотеками (libboost, OpenSSL), що часто призводить до конфліктів версій.
- **Для Go/Rust (Geth, Nethermind):** Процес більш автоматизований. Менеджери пакетів (Cargo або Go Modules) самостійно стягують залежності, а результатом є статичний бінарний файл, який легко переносити між системами.

2 ОСОБЛИВОСТІ РОЗГОРТАННЯ ТА КОНФІГУРАЦІЇ ВУЗЛІВ

Розгортання вузла – це створення вашої точки присутності в мережі. Вона відповідає за все: верифікацію, зберігання та трансляцію даних. На цьому етапі системному аналітику не можна помилятися. Слабке «залізо» або крива конфігурація призведуть до десинхронізації, а в гіршому випадку – до вразливості всієї приватної мережі.

2.1. Системні вимоги: де зазвичай «вузьке місце»?

При плануванні інфраструктури важливо розрізняти обіцянки з документації та реальні потреби системи для стабільної роботи 24/7. Основний удар зазвичай тримають три компоненти: диски, пам'ять і мережа.

1. Дискова підсистема: битва за IOPS

Це, мабуть, найкритичніший вузол для будь-якої ноди.

- **Ethereum (Geth/Nethermind):** Через структуру «дерева станів» система постійно робить тисячі дрібних операцій читання/запису. Використовувати старі HDD для Full Node – ідея марна, вони просто не витягнуть потрібну кількість IOPS. Аналітику варто відразу закладати SSD (краще NVMe) обсягом від 1 ТБ для Mainnet або від 100 ГБ для приватної мережі.
- **Bitcoin, Litecoin, Dash:** Вони не такі вибагливі до швидкості, бо модель UTXO дозволяє краще кешувати дані. Але обсяг блокчейну невинно зростає. Краще заздалегідь планувати дискові масиви з можливістю гарячого розширення.

2. Оперативна пам'ять: скільки не шкода?

- **Bitcoin Core:** Вузол «заведеться» навіть на 2 ГБ RAM, якщо вимкнути графічний інтерфейс. Але для швидкої реіндексації краще мати хоча б 4–8 ГБ.
- **Ethereum:** Тут пам'ять потрібна для «гарячих» гілок дерева станів. У Geth є параметр `--cache`, який дозволяє виділити RAM під базу даних. Для нормальної роботи 16 ГБ RAM уже стали фактичним стандартом. Інакше ви отримаєте постійний «disk thrashing» (надмірне звернення до диска).

3. Програмні середовища та віртуалізація

- **Linux (Ubuntu/Debian):** Це база. Майже вся документація та скрипти автоматизації (Ansible, Docker) написані під Linux.
- **Windows та WSL:** Оптимальний вибір для аналітиків, які не хочуть покидати Windows. WSL дозволяє запускати ноди в нативному Linux-оточенні без великих накладних витрат. Продуктивність файлової системи при цьому залишається на висоті.
- **Docker:** Ключ до створення тестових мереж (DevNets). Контейнери дозволяють ізолювати різні версії бібліотек і миттєво розгортати ідентичні вузли в хмарі.

4. Мережеві вимоги

Не забуваємо про трафік. Публічний вузол може «з'їдати» від 200 ГБ до 1 ТБ на місяць. У приватних мережах головне – Latency (затримка). Якщо блок передається між вузлами довше, ніж він генерується, мережа почне розвалюватися на форки. Чи готові ви до такого сценарію? Навряд чи.

Детальні системні вимоги

При проектуванні інфраструктури критично важливо враховувати "вузькі місця" кожної системи, особливо щодо дискової підсистеми та об'єму RAM.

Платформа	CPU (Ядра)	RAM	Тип диску (Storage)	Особливості
Bitcoin	2+ (x86-64/ARM)	4–8 GB	500 GB+ HDD/SSD	Невимогливий до швидкості диска завдяки моделі UTXO.
Ethereum	4+ (Fast)	16 GB+	1 TB+ NVMe SSD	Потребує високого IOPS для оновлення дерева станів Merkle Patricia.
Dash Masternode	2+	4–8 GB	100 GB+ SSD	Вимагає заставу в 1000 DASH та статичну IP-адресу.

Порівняння інтенсивності IOPS при синхронізації

Основна відмінність у навантаженні на диск виникає через архітектуру стану мережі. Ethereum змушує диск працювати у режимі постійного довільного читання/запису малих об'єктів.

- **Ethereum (Geth):** Графік демонструє екстремально високу інтенсивність IOPS через необхідність постійного оновлення "дерева станів" (Merkle Patricia Tree). Використання звичайних HDD призводить до десинхронізації вузла.
- **Bitcoin:** Навантаження на диск є більш лінійним та передбачуваним. Основна активність припадає на запис нових блоків та індексацію набору UTXO, що дозволяє системі стабільно працювати навіть на менш швидкісних накопичувачах.

2.2. Процес ініціалізації приватної мережі: від генезис-блоку до перших транзакцій

Для системного аналітика ініціалізація – це, власне, момент істини. Саме тут ми визначаємо базові правила гри: від того, наскільки складно буде видобувати блоки, до початкового розподілу активів. Якщо в публічних мережах (Mainnet) ці параметри викарбувані в коді «на віки», то розгортання приватного середовища дає нам повну свободу. Але за умови, що ми розуміємо

конфігураційні механізми. А вони в Ethereum та Bitcoin-подібних системах відрізняються кардинально.

2.2.1. Ініціалізація Ethereum та роль файлу Genesis.json

Ethereum тут стоїть особно. Йому потрібне явне визначення початкового стану мережі через спеціальний файл – Genesis.json. Це такий собі «паспорт» майбутньої мережі. Якщо проігнорувати цей крок, клієнт за звичкою кинеється шукати пірів у публічній мережі й почне викачувати сотні гігабайт зайвих даних.

Що ми прописуємо в Genesis.json? Варто зауважити кілька ключових полів:

- **config:** Тут живе chainId – унікальний код мережі. Він критично важливий для захисту від атак повторного відтворення. Також тут вказуємо посилання на хардфорки (London, Berlin тощо).
- **difficulty:** Наскільки важко «підписати» блок. Для приватних тестів ми зазвичай викручуємо цей параметр на мінімум. Навіщо чекати? Нехай звичайний процесор клацає блоки миттєво.
- **gasLimit:** Це стеля обчислювальної потужності на один блок. Важливо для складних смарт-контрактів, які іноді просто «не лізуть» у стандартні ліміти публічної мережі.
- **alloc:** Дозволяє «надрукувати» ефір на потрібні адреси прямо в першому блоці. Це позбавляє від нудного майнінгу заради тестових коштів.

Сам процес запускається командою `geth init path/to/genesis.json`. Вона створює локальну базу даних (LevelDB), яка й стає фундаментом. Примітно, що розробники на Windows часто роблять це через WSL – так простіше уникнути проблем із файловими шляхами та системними викликами, які в Linux працюють «як треба».

Приклад структури genesis.json для аналітика:

```
{
  "config": { "chainId": 1234, "homesteadBlock": 0 },
  "difficulty": "0x1",
  "gasLimit": "0x8000000",
  "alloc": { "0x...": { "balance": "1000000000000000" } }
}
```

2.2.2. Ініціалізація Bitcoin-подібних систем (Litecoin, Dash) через режими тестування

З Bitcoin та його родичами (Litecoin, Dash) історія зовсім інша. Замість зовнішніх JSON-файлів ми маніпулюємо режимами запуску самого демона та параметрами у `bitcoin.conf`.

Найбільш ходовий варіант для приватної розробки – режим **Regression Test (regtest)**. Мережа ініціалізується з нульовою складністю. Це ідеально: ви можете

самі генерувати блоки за запитом через команду `generatetoaddress`. Ніякого очікування – транзакції підтверджуються миттєво.

У файлі `bitcoin.conf` зазвичай налаштовуємо таке:

- **rpcuser та rpcpassword:** Щоб можна було стукатися до вузла через API.
- **listen:** Дозвіл на вхідні з'єднання. Без цього ваша приватна мережа залишиться мережею з одного вузла.
- **server:** Активація JSON-RPC інтерфейсу. Без нього `bitcoin-cli` просто не зможе віддати жодної команди.

Щодо Dash, то тут є свій нюанс – Мастерноди. Їх треба ініціалізувати окремо: створювати приватні ключі та вносити заставу (collateral) у 1000 DASH. В ізольованому `regtest`-середовищі ці гроші зазвичай просто майняться за кілька хвилин.

2.2.3. Спеціалізована ініціалізація в NEO

NEO вимагає, мабуть, найбільш прискіпливого підходу до учасників. Тут ми створюємо файл `protocol.json`, де чітко прописуємо адреси «вузлів-консенсусу». Це важливий момент: на відміну від Ethereum, де майнити може будь-хто, у NEO право голосу мають лише обрані. Тому аналітику доводиться збирати публічні ключі всіх учасників ще до того, як з'явиться перший блок.

2.2.4. Порівняльний аналіз підходів

Якщо подивитися на це збоку, то Ethereum виглядає більш гнучким конструктором для створення «світового стану» з нуля. Ви можете задати будь-які правила через Genesis-файл. Bitcoin Core у цьому плані консервативніший. Він спирається на жорстко вбудовані режими (`testnet/regtest`). Щоб змінити фундаментальні правила в Bitcoin, довелося б переписувати сам код клієнта, а не просто правити конфіг.

2.3. Мережева топологія: як вузли шукають один одного

Мережевий рівень блокчейну – це чистий P2P. Кожен вузол тут і жнець, і швець, тобто і сервер, і клієнт одночасно. Але як вони знаходять один одного в темряві інтернету? Механізми пошуку в Ethereum та Bitcoin-системах різняться, що прямо впливає на те, як ми налаштовуємо фаєрволи.

2.3.1. Ethereum: Node Discovery та протокол Kademlia

Ethereum використовує досить сучасний підхід – протокол Kademlia та структуровані таблиці хешів (DHT). Кожен вузол має свій унікальний Node ID. Щоб знайти «сусідів», вузол розсилає UDP-пакети та міряє «відстань» (так звана XOR-метрика) до інших учасників. Примітно, що така мережа дуже живуча: вона сама перезбирається, навіть якщо половина учасників раптово зникне.

Але є нюанс. У приватних мережах автоматичний пошук не спрацює без початкової точки входу. Тут аналітику потрібно розгорнути **Bootnodes** – вузли зі

статичними IP. Їхні адреси (enode-посилання) ми прописуємо іншим учасникам при запуску. Без «бутнодів» вузли просто не побачать один одного в ізоляції.

Зазвичай Geth працює на порту 30303. Пам'ятайте: цей порт має бути відкритий і для TCP (передача даних), і для UDP (пошук вузлів).

2.3.2. Bitcoin, Litecoin та Dash: Традиційний P2P та DNS-seeds

Системи на базі Bitcoin Core простіші, але надійні. При першому запуску вузол іде до **DNS-seeds**. Це сервери, які просто віддають список активних IP-адрес. Вузол записує їх у peers.dat і далі підключається вже напряму.

Тут усе тримається на TCP-з'єднаннях. Вузли перекидаються короткими повідомленнями: наприклад, inv (інвентар), щоб сказати «гей, у мене є новий блок», і getdata, щоб цей блок отримати. Порти у кожного свої: 8333 для Bitcoin, 9333 для Litecoin та 9999 для Dash. На відміну від Ethereum, тут зазвичай не треба возитися з UDP для пошуку пірів, що спрощує життя системному адміністратору.

2.3.3. Дворівнева топологія Dash

Dash додає ще один рівень складності. Його мережа ієрархічна. На першому поверсі – звичайні вузли, на другому – Мастерноди. Останні мають постійно тримати TCP-зв'язок між собою, щоб формувати кворуми для миттєвих транзакцій (InstantSend). Для аналітика це означає одне: Мастернодам потрібні стабільні, статичні IP-адреси. Змінили IP? Вузол вилітає з активного списку. Тут доводиться ретельно планувати стабільність каналів або використовувати статичні пули адрес.

2.3.4. Налаштування в корпоративних мережах

Коли ми виводимо блокчейн у реальну корпоративну інфраструктуру, починаються «танці з бубном» навколо трьох речей:

1. **NAT та Port Forwarding:** Якщо вузол за NAT, він зможе підключатися до інших, але до нього – ніхто. Це робить мережу крихкою. Для лікування використовуємо параметри extip (в Bitcoin) або nat (в Ethereum).
2. **Безпека RPC-порту:** Це критично. Порти управління (8545 для ETH, 8332 для BTC) ніколи не мають дивитися в інтернет. Це все одно що залишити ключі від сейфа в дверях.
3. **Ізоляція:** Для приватних проєктів краще використовувати VPN або окремі VLAN. Навіщо вам змішувати трафік тестування з публічним шумом?

3 КОМПОНЕНТНИЙ АНАЛІЗ ТА МЕХАНІЗМИ КОНСЕНСУСУ

3.1. Модуль консенсусу: технічне порівняння PoW, PoS та dBFT

Консенсус у розподілених системах – це значно більше, ніж просто спосіб «домовитися» про наступний блок. По суті, ми маємо справу зі складним протоколом забезпечення візантійської відмовостійкості (BFT). Для системного аналітика вибір конкретного алгоритму стає критичним питанням. Чому? Бо саме він визначає «вартість атаки», швидкість фіналізації транзакцій і, що важливо, тип необхідної інфраструктури – чи будемо ми вкладатися в гігават-години електрики, чи в «заморожений» капітал.

3.1.1. Proof-of-Work (PoW): логіка обчислювальної складності

В основі PoW лежить проста, на перший погляд, ідея: використання реального фізичного ресурсу для захисту віртуального реєстру.

- **Математична основа:** У таких системах, як Bitcoin, Litecoin або Dash, майнінг – це не що інше, як пошук значення Nonce. Воно має бути таким, щоб разом із даними блоку видати хеш, менший за цільове значення (Target). Процес суто ймовірнісний. Ніяких хитрощів, тільки повний перебір варіантів.
- **Порівняння алгоритмів хешування:**
 - **SHA-256 (Bitcoin):** Тут використовується каскадне подвійне хешування. Оскільки логічні операції (AND, XOR, ROT) дуже прості, алгоритм швидко переїхав на ASIC-чіпи. Примітно, що це призвело до тотальної централізації «заліза». Аналітик має розуміти: сьогодні майнити Bitcoin на звичайному процесорі технічно неможливо.
 - **Scrypt (Litecoin):** Цю функцію задумували як «memory-hard», тобто таку, що вимагає багато пам'яті. Логіка була в тому, щоб RAM на чіпі коштувала дорожче за обчислювальні ядра, стримуючи появу ASIC. Проте варто зауважити, що це автоматично підвищує вимоги до затримок пам'яті при розгортанні валідаторів.
 - **X11 (Dash):** Мабуть, найбільш стійкий підхід. Дані послідовно проходять через 11 різних хеш-функцій. Якщо одну зламають – блокчейн встоїть. До того ж такий ланцюжок функцій менше гріє GPU, що важливо для енергоефективності.
- **Регулювання складності (Difficulty Adjustment):** Це критичний момент для приватної мережі. У Bitcoin складність перераховується раз на 2016 блоків (приблизно кожні два тижні). Dash натомість використовує Dark Gravity Wave – перерахунок іде на кожному блоці. Що це дає аналітику? Приватна мережа Dash адаптується до стрибків потужності миттєво.

3.1.2. Proof-of-Stake (PoS): еволюція Ethereum (The Merge)

Після історичного оновлення Ethereum перейшов на модель, де безпеку гарантує економічний ресурс – застава (stake).

- **Архітектура Beacon Chain:** Тепер при розгортанні вузла ми маємо справу з двома модулями. Перший – Execution Engine (EE), який «крутить» смарт-контракти. Другий – Consensus Engine (CE), наприклад Prysm, що керує логікою PoS.
- **Slashing та Rewards:** У PoW помилка майнера – це лише дарма спалена електрика. У PoS все жорсткіше. Шкідлива активність карається «спалюванням» частини застави. При налаштуванні кластерів високої доступності (HA clusters) аналітик має бути вкрай обережним, щоб не отримати штраф за «подвійний підпис» через банальну помилку конфігурації.
- **Slots & Epochs:** Час у PoS нарізаний на слоти по 12 секунд. 32 слоти складають епоху. Це робить час підтвердження транзакцій набагато передбачуванішим, ніж у Bitcoin.

3.1.3. dBFT (Delegated Byzantine Fault Tolerance) у NEO

Алгоритм NEO найближче стоїть до класичних теорій розподілених систем.

- **Ролі учасників:** Мережа чітко розділена. Є звичайні користувачі, а є професійні «бухгалтери» (Bookkeepers). Спікер пропонує блок, а інші мають підтвердити його двома третинами голосів.
- **Абсолютна фінальність (Finality):** Це, мабуть, головний козир для аналітика. У Bitcoin чи Ethereum фінальність ймовірна – завжди є мізерний шанс, що з'явиться довша гілка. У NEO блок остаточний одразу після голосування. Для банківських транзакцій це критично.
- **Ризики:** Оскільки кількість вузлів консенсусу обмежена, розгортання вимагає суворого контролю. Вихід з ладу навіть кількох учасників може просто «покласти» всю генерацію блоків.

3.2. Віртуальні машини та середовища виконання

Якщо консенсус вирішує, *коли* блок потрапить до реєстру, то середовище виконання (Runtime Environment) визначає, *яку саме* роботу виконає вузол. Важливо розуміти прірву між Тюрінг-повними системами (Ethereum, NEO) та декларативними сценаріями (Bitcoin, Script). Це і є межа можливостей платформи.

3.2.1. Ethereum Virtual Machine (EVM): квазі-Тюрінг-повний «світовий комп'ютер»

EVM – це стекова віртуальна машина, що виконує байт-код на кожному вузлі. Це така собі «пісочниця» (sandbox), ізольована від мережі та файлової системи хоста.

- **Специфіка:** Машина оперує 256-бітними словами, що ідеально для Кессак-256. Вона має три типи пам'яті: Stack для обчислень, Memory (тимчасова) та Storage (дорога та вічна).
- **Механізм Газу (Gas):** Оскільки код може містити нескінченні цикли, виникає проблема зупинки. Ethereum вирішує це через «Газ».

Користувач платить за кожну операцію. Скінчився газ – транзакція скасувалася. Під час розгортання приватної мережі аналітик повинен дуже тонко налаштувати targetGasLimit, щоб важкі корпоративні контракти працювали, але не стали інструментом DoS-атаки.

- **Детермінізм:** Це залізне правило. Результат на всіх вузлах має бути ідентичним, незалежно від того, яка там стоїть ОС.

3.2.2. Bitcoin Script: безпека через обмеженість

Bitcoin, Litecoin та Dash використовують Script. Мова навмисно примітивна: у ній немає циклів.

- **Логіка:** Це послідовність команд (OpCodes), що виконуються зліва направо. Чому це добре? Бо час виконання будь-якої транзакції передбачуваний. Це робить вузли надзвичайно стійкими до атак на процесор. Хоча Script обмежений, на ньому цілком можна будувати мультипідписи або канали оплати на кшталт Lightning Network.
- **Dash:** Ця мережа додає свої інструкції для InstantSend, що фактично є надбудовою над базовим Script.

3.2.3. NeoVM: орієнтація на корпоративну розробку

NEO пропонує підхід, що кардинально відрізняється від Ethereum.

- **Багатомовність:** На відміну від Ethereum з його Solidity, NeoVM розуміє C#, Java, Python та Go. Компілятори просто перетворюють знайомий код на байт-код машини.
- **Продуктивність:** Завдяки JIT-компіляції (Just-In-Time), NeoVM працює швидше за EVM. Код смарт-контрактів тут має прямий доступ до системних активів та ідентифікації, що робить бізнес-логіку цілісною.

3.2.4. Що ж обрати аналітику?

Підсумовуючи, ми маємо три різні філософії. Ethereum пропонує максимальну гнучкість, але вимагає платити за складність (і ризикувати через Reentrancy-атаки). Bitcoin та Litecoin – це про безпеку та передбачуваність, де ми свідомо жертвуємо складністю. NEO – це швидкість і зручність для традиційного бізнесу, але з іншим підходом до фінальності та більш закритою екосистемою.

Критерій	Ethereum (EVM)	Bitcoin (Script)	NEO (NeoVM)
Тюрінг-повність	Так (Квазі)	Ні	Так
Основні мови	Solidity, Vyper	Script (OpCodes)	C#, Java, Python, Go

Захист від циклів	Механізм Газу	Відсутність циклів	Механізм системних комісій
Складність дебагу	Висока	Низька	Середня

3.3. Порівняння характеристик та аналіз ключових метрик

Для системного аналітика зведена таблиця – це не просто набір цифр «для довідки», а робочий інструмент. Саме тут ми бачимо фундамент для архітектурних рішень. Кожен параметр визначає, чи підійде конкретний блокчейн для вашого завдання: чи то швидкі мікроплатежі, чи то неповороткі, але надійні корпоративні системи.

3.3.1. Зведена аналітична таблиця характеристик

Параметр порівняння	Bitcoin (BTC)	Ethereum (ETH)	Dash (DASH)	NEO (NEO)	Litecoin (LTC)
Архітектурна модель	Монолітна, UTXO	Монолітна (Merge), Accounts	Дворівнева, UTXO	Модульна, Accounts	Монолітна, UTXO
Механізм консенсусу	PoW (SHA-256)	PoS (Gasper)	PoW (X11) + PoS (MN)	dBFT	PoW (Scrypt)
Середній час блоку	10 хвилин	~12–14 секунд	2,5 хвилини	15–20 секунд	2,5 хвилини
Пропускна здатність	5–7 транз./сек.	15–30 транз./сек.	~35–56 транз./сек.	до 1000+ транз./сек.	~56 транз./сек.
Фіналізація транз.	Ймовірність на (~60 хв)	Ймовірність на (~6–12 хв)	Миттєва (InstantSend)	Абсолютна (1 блок)	Ймовірність на (~30 хв)
Рівень програмованості	Низький (Script)	Високий (Solidity/EVM)	Середній	Високий (C#, Java, Python)	Низький (Script)

Вартість розгортання	Низька (вузол)	Висока (SSD/32 ETH)	Середня (+1000 DASH)	Середня	Низька (вузол)
-----------------------------	----------------	---------------------	----------------------	---------	----------------

3.3.2. Часові характеристики та латентність (Latency)

Коли ми розгортаємо мережу, нас найбільше цікавить одне: коли транзакцію можна вважати незворотною?

Примітно, що у Bitcoin та Litecoin через саму природу PoW завжди є ризик появи паралельного ланцюга. Тож доводиться чекати. У Bitcoin безпечним вважається очікування 6 підтверджень. А це, на хвилинку, ціла година. Очевидно, що для роздрібної торгівлі, де оплата має бути миттєвою, це зовсім не варіант.

Ethereum після переходу на PoS намагався виправити ситуацію через «чекпоінти» (Finality). Але й тут процес триває близько 12 хвилин.

Цікаво, що найкращу латентність видають NEO та Dash. У Dash функція InstantSend дозволяє Мастернодам блокувати входи транзакції за лічені секунди. Подвійна витрата стає неможливою ще до того, як транзакція потрапить у блок. У NEO все ще простіше: завдяки dBFT фінальність настає миттєво після першого ж підтвердження. Для систем автоматизованого трейдингу це часто стає вирішальним фактором.

3.3.3. Масштабованість та пропускна здатність (Throughput)

Тут важливо розуміти межі. Bitcoin свідомо тримає невеликий розмір блоку (до 4 МБ через SegWit). Навіщо? Щоб мережа лишалася децентралізованою і вузол можна було запустити навіть на «слабкому» залізі.

NEO йде іншим шляхом. Вони жертвують децентралізацією – тримають лише кілька вузлів консенсусу, зате розганяють швидкість до рівня VISA.

А що ж Ethereum? Він шукає баланс. Але на базовому рівні (Layer 1) пропускна здатність усе одно низька. Тому аналітикам доводиться дивитися в бік рішень другого рівня (Layer 2), якщо проект масштабний.

3.3.4. Апаратні та капітальні витрати на розгортання

Коли ми готуємо ТЗ на розгортання, витрати зазвичай ділимо на дві категорії.

Перша – технічна. Як вже було згадано, вимоги до швидкості SSD у Ethereum та NEO на порядок вищі, ніж у консервативного Bitcoin.

Друга – економічна. Це, якщо хочете, «вхідний квиток». Хочете запустити Мастерноду Dash? Готуйте заставу в 1000 монет. У Ethereum для самостійної валідації потрібно 32 ETH. Це серйозний бар'єр. З іншого боку, Bitcoin та Litecoin дозволяють піднімати вузли взагалі без вкладень. Це ідеально для аудиту чи некомерційних тестів.

3.3.5. Безпека та хеш-алгоритми

Вибір алгоритму – це питання стійкості до зламу.

Використання X11 у Dash робить систему живучою: навіть якщо один криптографічний примітив «впаде», інші втримають блокчейн.

Script у Litecoin дуже «любить» оперативну пам'ять. При розгортанні великих кластерів доведеться ретельно планувати серверні стійки.

Варто зауважити, що Ethereum, перейшовши на Gasper, взагалі перевів безпеку з площини обчислювального «заліза» у площину теорії ігор. Тепер усе залежить від економічних стимулів.

4 АНАЛІЗ ВЗАЄМОЗАМІННОСТІ МОДУЛІВ ТА ІНТЕГРАЦІЇ

Чи можна замінити базу даних, не чіпаючи консенсус? Або оновити мережу, не переписуючи смарт-контракти? Це питання модульності.

4.1. Гнучкість модульної архітектури

Сьогодні в системному проектуванні цінується високий рівень абстракції. Блокчейн-клієнт – це такий собі «лишковий пиріг» із мережевого, консенсусного та прикладного шарів. Але «зчеплення» (coupling) між цими шарами у всіх різне.

4.1.1. Ethereum: еталон модульності через Engine API

Після The Merge Ethereum став найбільш гнучким конструктором.

Архітектура чітко розрізана на Execution Layer (EL) та Consensus Layer (CL). Вони «спілкуються» через стандартний Engine API. Що це дає нам на практиці? Ви можете взяти Geth як двигун виконання, а Prysm – як клієнт консенсусу. Або замінити Geth на Nethermind, якщо ваша інфраструктура заточена під .NET. Така взаємозамінність рятує від "vendor lock-in" і дозволяє в разі критичного багу переключитися на іншу реалізацію за лічені хвилини.

4.1.2. Монолітна архітектура Bitcoin, Litecoin та Dash

Тут ситуація інша. Системи на базі Bitcoin Core – це переважно моноліти.

Код для перевірки скриптів жорстко «прошитий» разом із логікою блоків і мережевим стеком. На перший погляд, це надійно. Але спробуйте замінити стандартну LevelDB на більш швидку RocksDB – вам доведеться лізти в самі нетрі сирцевого коду і перезбирати весь проект.

Щоправда, Dash вносить трохи різноманітності через рівень Мастернод. Це така собі «надбудова», де можна оновлювати сервіси (як-от Dash Drive), не ламаючи базовий PoW-рівень.

4.1.3. Модульність рівнів збереження даних (Persistence Layer)

Оптимізація дискових операцій (I/O) – це часто головний біль аналітика.

Більшість використовує LevelDB від Google. Але під навантаженням вона часто стає тим самим «пляшковим горлом».

В Ethereum (зокрема в клієнті Erigon) цю проблему вирішили через MDBX. Доступ до стану мережі там значно швидший. У NEO ж архітектура дозволяє використовувати плагіни, що дуже зручно, якщо ви хочете «подрудити» приватний блокчейн із корпоративними SQL або NoSQL базами.

4.1.4. Абстракція мережевого стеку

Ethereum активно впроваджує Libp2p, що дозволяє легко додавати нові протоколи, як-от QUIC.

Bitcoin-подібні системи в цьому плані консервативні. У них власний протокол повідомлень. Спроба замінити його на щось сучасне просто зробить ваш вузол несумісним з іншою мережею. Це робить їх найбільш закритими для модифікацій на мережевому рівні.

Підсумуємо: Якщо проект вимагає частих оновлень або специфічних баз даних – ваш вибір Ethereum або NEO. Якщо ж потрібна максимальна стабільність і «загартований» код, де зміни вносяться раз на п'ять років – Bitcoin та Litecoin залишаються поза конкуренцією.

4.2. Інтерфейси програмування (API) та інтеграція: від JSON-RPC до нативних оракулів

Якщо з модульністю ми розібралися як із внутрішнім «залізом», то API – це те, як система спілкується із зовнішнім світом. Системний аналітик має тверезо оцінити: наскільки реально «подружити» обраний вузол із банківським софтом, мобільним додатком або, скажімо, датчиками IoT?

4.2.1. Стандартизація через JSON-RPC: спільне та відмінне

Цікаво, що всі п'ять систем – BTC, ETH, LTC, DASH та NEO – тримаються одного стандарту JSON-RPC. Щоправда, диявол, як завжди, у деталях.

- **Bitcoin, Litecoin, Dash:** тут ми бачимо класичний набір методів на кшталт `getblockchaininfo` чи `sendtoaddress`. Працює все за принципом «запит-відповідь». Варто зауважити: стандартний RPC-сервер Bitcoin Core не надто любить натовпи користувачів. Якщо у вас тисячі одночасних запитів, без проксі-шарів (як-от Bitcore Wallet Service) не обійтися.
- **Ethereum (Geth/Nethermind):** пропонує набагато ширший простір імен – `eth_`, `web3_`, `net_` та `debug_`. Примітно, що тут є підтримка GraphQL. Для аналітика це знахідка: можна витягнути всі транзакції конкретного контракту одним запитом. Це економить трафік і нерви.
- **NEO:** на додачу до стандартів, хлопці пропонують дуже зручні SDK. З блокчейном працюєш так само легко, як зі звичайною базою даних.

4.2.2. Поточкова передача даних: WebSockets та ZeroMQ (ZMQ)

Коли треба стежити за ринком або ловити фрод у реальному часі, звичайні запити – це вчорашній день.

- **Ethereum Pub/Sub:** через WebSockets можна підписатися на події (`newHeads`, `logs`). Це критично для аналізу, бо дозволяє миттєво «чути» виклики смарт-контрактів.
- **Dash та ZeroMQ:** Dash (як і Bitcoin) підтримує ZMQ – це така високопродуктивна черга повідомлень. Додаток просто «слухає» мережу. Ніякого постійного опитування вузла. Мінімум латентності.

4.2.3. Проблема Оракулів: нативні рішення проти зовнішніх

Будь-який блокчейн «сліпий» від народження – він не знає ні про курси валют, ні про погоду за вікном. Як ми це вирішуємо?

В Ethereum доводиться кликати на допомогу сторонні сервіси на кшталт Chainlink. Аналітик має одразу закладати в проект витрати на оракулів і враховувати ризики їхньої централізації.

У NEO ситуація інакша. Тут є нативна служба оракулів – це унікальна фіча. Смарт-контракти роблять HTTP-запити до зовнішніх API безпосередньо. Архітектура стає простішою, бо вам не треба городити складні зовнішні підпорки.

4.2.4. Cross-chain інтеграція та інтероперабельність

Давайте будемо чесними: сьогодні рідко хто працює лише в одній мережі.

- **Atomic Swaps:** підтримуються в Bitcoin, Litecoin та Dash. Можна міняти активи між мережами без жодних посередників.
- **Poly Network (NEO):** NEO є частиною великої мережі інтероперабельності. Можна легко перекидати дані між NEO та Ethereum. Що це дає аналітику? Можна будувати гібриди: розрахунки в Bitcoin, а складна логіка – в NEO.

4.2.5. Безпека інтеграційного шару

RPC-інтерфейс треба берегти як зіницю ока. Важливо відобразити у звіті три пункти:

1. **RPC-Auth:** тільки міцні паролі та хешування.
2. **IP Whitelisting:** пускаємо до вузла лише довірені сервери.
3. **SSH Tunneling / TLS:** JSON-RPC передає все у відкритому вигляді, тому шифрування трафіку – це не побажання, а обов'язок.

4.3. Аналіз масштабованості та оновлюваності: форки, управління та вектори розвитку

Чи зможе система витримати наплив даних через рік? Як вона оновлюється? Це питання «трилеми блокчейну» – як балансувати між безпекою, швидкістю та децентралізацією.

4.3.1. Масштабованість: Layer 1 проти Layer 2

Ми бачимо два абсолютно різні підходи до «розгону» мережі.

Перший – **вертикальний (On-chain)**. NEO та Dash роблять ставку на потужність. У NEO це працює через обмежену кількість сильних вузлів. Dash використовує Мастерноди для підтримки блоків до 2 МБ, щоб більше транзакцій

влязило в базу. Bitcoin та Litecoin ідуть консервативним шляхом: блоки малі, щоб вузол завівся навіть на старому ноутбучі.

Другий – **горизонтальний (Off-chain)**. Тут ми виносимо мікроплатежі за межі мережі. У Bitcoin це Lightning Network. Ethereum же обрав «Rollup-centric» модель – основний ланцюг лише фіксує результати, а вся робота йде на другому рівні (Optimistic або ZK-Rollups). Архітектура стає складнішою через містки, і це треба враховувати.

4.3.2. Механізми управління (Governance): хто тут головний?

Процес оновлення правил може бути як тихим голосуванням, так і гучним скандалом.

- **Off-chain Governance (BTC, LTC, ETH):** все тримається на консенсусі спільноти та розробників. Є документи BIP чи EIP. Не домовилися? Отримуємо Hard Fork, як це сталося з Bitcoin Cash.
- **On-chain Governance (Dash, NEO):** у Dash є крута штука – Treasury. 10% винагороди за блок іде у фонд, а власники Мастернод голосують, на що ці гроші витратити. Це робить систему самодостатньою. У NEO є Council – власники tokenів обирають раду, яка може змінювати комісії навіть без переписування коду.

4.3.3. Проблема розростання стану (State Bloat)

Рано чи пізно кожен блокчейн «жиріє». Даних стає забагато.

Ethereum активно готує фази The Verge та The Purge, щоб вузли не тягнули за собою всю історію завдяки деревам Веркла. Bitcoin пропонує Pruning (обрізку): зберігаємо лише поточні баланси (UTXO), а старе сміття видаляємо. Важливо пам'ятати: такий вузол не підійде, якщо вам потрібен повний аудит історії з першого дня.

4.3.4. Порівняння гнучкості оновлень (Forks)

Тип оновлення	Bitcoin / Litecoin	Ethereum	Dash	NEO
Soft Fork	Пріоритет (SegWit)	Рідко	Так	Майже ніколи
Hard Fork	Вкрай рідко	Регулярно	Регулярно	Через Council
Ризик розколу	Високий	Середній	Низький	Мінімальний

Підсумовуючи: NEO та Dash – найбільш гнучкі завдяки вбудованим «виборам». Ethereum летить вперед на шаленій швидкості через регулярні хардфорки. А Bitcoin? Він лишається скелею. Консервативність – це його

головна фішка як «цифрового золота». Тут стабільність правил цінують більше за будь-які нові «фішки».

5 ВИСНОВКИ ТА СТРАТЕГІЧНІ РЕКОМЕНДАЦІЇ ЩОДО ВИБОРУ ПЛАТФОРМИ

Підсумовуючи наш аналіз, варто зауважити, що ринок блокчейн-протоколів нарешті перейшов у фазу технологічної зрілості. Кожна система вже не просто «існує», а займає цілком конкретну нішу. На основі проведеного порівняння ми сформулювали зрозумілі критерії, які допоможуть визначитися з платформою для розгортання як публічних, так і приватних рішень.

5.1. Підсумкова оцінка технологічної придатності

Примітно, що кожна з п'яти систем по-своєму балансує на гранях відомої «трилеми блокчейну»:

1. **Bitcoin та Litecoin:** це такі собі «консервативні моноліти». Їхня архітектура – найбільш загартована в боях (battle-tested), але водночас і найменш гнучка. Чи погано це? Зовсім ні, якщо ваш проект передбачає використання блокчейну суто як незмінного реєстру вартості. Високий поріг входження через специфіку Bitcoin Script тут компенсується максимальною передбачуваністю.
2. **Ethereum:** сьогодні це повноцінна «динамічна екосистема». Після переходу на PoS і розділення на рівні виконання та консенсусу, він став стандартом для децентралізованого світу. Щоправда, за це доводиться платити: висока вартість експлуатації та «важкі» вузли роблять його не найкращим вибором для скромних приватних мереж.
3. **Dash:** справжній «прагматичний інструмент». Дворівнева структура з Мастернодами елегантно вирішує питання швидкості та фінансової незалежності завдяки вбудованому бюджету (Treasury). Що важливо, це «золота середина» для фінансових сервісів, де критично потрібен InstantSend.
4. **NEO:** найбільш «бізнес-орієнтована» платформа. Завдяки тому, що NeoVM розуміє звичні мови програмування, а в систему вже «з коробки» зашиті оракули, інтегрувати такий блокчейн у корпоративне середовище можна без зайвого головного болю та перенавчання штату.

5.2. Стратегічна матриця вибору для системного аналітика

Щоб спростити процес прийняття рішень, ми звели бізнес-задачі та відповідні їм технологічні стеки в одну таблицю:

Бізнес-сценарій	Рекомендована система	Ключовий аргумент
Зберігання капіталу / Резервний реєстр	Bitcoin (BTC)	Гранична стійкість мережі та світове визнання.

Складні dApps / DeFi / NFT проекти	Ethereum (ETH)	Величезна база готових бібліотек і спільнота розробників.
Швидкі роздрібні платежі (PoS-термінали)	Dash / Litecoin	Майже нульові комісії та миттєва фінальність у Dash.
Корпоративні мережі (Private Chain)	NEO / Ethereum	Гнучке налаштування генезис-блоку та потужні смарт-контракти.
Системи з реальними даними (IoT / Supply Chain)	NEO	Наявність вбудованих оракулів для HTTP-запитів до зовнішнього світу.

5.3. Рекомендації щодо впровадження та мінімізації ризиків

На перший погляд, вибір платформи – це лише технічне питання, але як аналітик ви маєте пам'ятати про «підводні камені»:

- **Оцінка TCO (Total Cost of Ownership):** Вартість розгортання – це не тільки рахунки за сервери. Сюди ж додаємо витрати на моніторинг (особливо актуально для Мастернод Dash), «спалений» газ в Ethereum та постійну підтримку клієнтів у актуальному стані, щоб не випасти з мережі після чергового хардфорку.
- **Ризик технічного боргу:** Занадто захопившись специфічними фішками (на кшталт хитрих скриптів у Litecoin), ви ризикуєте зробити систему «невийзною». У майбутньому міграція на іншу платформу стане болісною. Старайтеся триматися стандартизованих інтерфейсів JSON-RPC.
- **Безпека ключів:** Важливо розуміти, що жоден суперсучасний консенсус не врятує проект, якщо процедури зберігання приватних ключів адміністраторів мають дірки.

Фінальна порада: Не намагайтеся зробити блокчейн «срібною кулею» для всіх проблем. Якщо задачу можна вирішити звичайною базою даних типу PostgreSQL – вирішуйте її саме так. Це буде і швидше, і дешевше. Блокчейн доречний лише там, де вам справді потрібна децентралізована довіра та захист від цензури. Чи варта ваша задача такої ціни? Вирішувати вам.